

AI Capstone Final Project Report - Entry

Group 13

This project studies imitation-learning-based robotic manipulation in a Physical AI pipeline. The Entry-level cutlery arrangement task is implemented and evaluated in simulation using the course-provided Isaac Sim / Isaac Lab and LeRobot-based framework.

I. Task Description - Cutlery arrangement

For the Entry-level task, our group worked on the **Cutlery Arrangement** task in the dining room scenario. The scene contains a dining table, a plate, a fork, a knife, and a single-arm Franka robot. At the beginning of each episode, the fork and knife are placed on the table with randomized initial poses. The objective is to manipulate the two utensils and place them at the correct positions relative to the plate:

- The fork should be placed on the left side of the plate.
- The knife should be placed on the right side of the plate.
- Both objects should remain on the table and be placed close enough to the target plate region.
- The task should be completed within the maximum rollout horizon.

This task evaluates whether a visuomotor policy can learn a structured pick-and-place behavior from demonstration data. Although the task appears simple, it still requires several important robot learning capabilities: visual grounding of small objects, stable grasping of thin cutlery, correct side-aware placement, and robustness to different object initial poses.

The Entry-level task follows the course pipeline: real-world data collection with UMI, object pose extraction, reconstruction in simulation, finite-state-machine-based robot motion generation, policy training, and rollout evaluation.

II. Data Collection

A. Real-World Data Collection with UMI

For the Entry-level task, we used the Universal Manipulation Interface (UMI) to collect real-world demonstration data in the ED305 classroom. The UMI device was used as a handheld data collection interface. During collection, the demonstrator moved the UMI gripper around the dining table scene while the system recorded demonstration videos and object observations.

In the course pipeline, the real-world UMI demonstrations are not directly replayed as robot trajectories. Instead, the demonstrations are processed to extract object poses and reconstruct the corresponding scene configurations in Isaac Sim. After reconstruction, robot trajectories are generated by a

finite-state-machine-based planner. This design avoids directly transferring noisy human hand trajectories to the robot arm, which would be difficult because of tracking noise, object-pose estimation error, sim-to-real mismatch, collision handling, and differences between human motion and robot kinematics.

Our UMI data processing workflow was:

1. Collect real-world UMI demonstrations in the dining-room cutlery arrangement setup.
2. Run the UMI processing pipeline to estimate object poses from the collected demonstrations.
3. Verify whether the extracted poses are physically reasonable.
4. Convert the extracted poses into the object-pose format required by Isaac Sim.
5. Use the object poses as initial scene configurations for finite-state-machine-based robot demonstration generation.
6. Store the successful simulation demonstrations in LeRobot format for policy training.

For the Entry-level experiments, we used two object-pose sets:

Object-pose source	Number of object poses	Description
Original UMI object poses	84	Object poses collected from the original dining-room UMI data.
Augmented object poses	300	Augmented object-pose set used to increase layout diversity.

Table 1. Object-pose description of Entry-level experiments

The 84-pose object set was used as the low-data baseline. The 300-pose object set was used as the main dataset for the final Entry-level experiments.

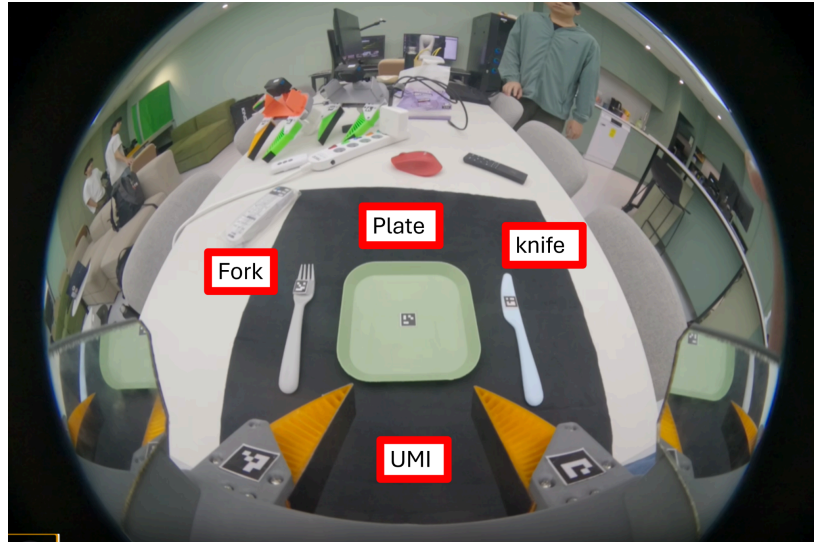


Figure 1. Real-world UMI collection scene

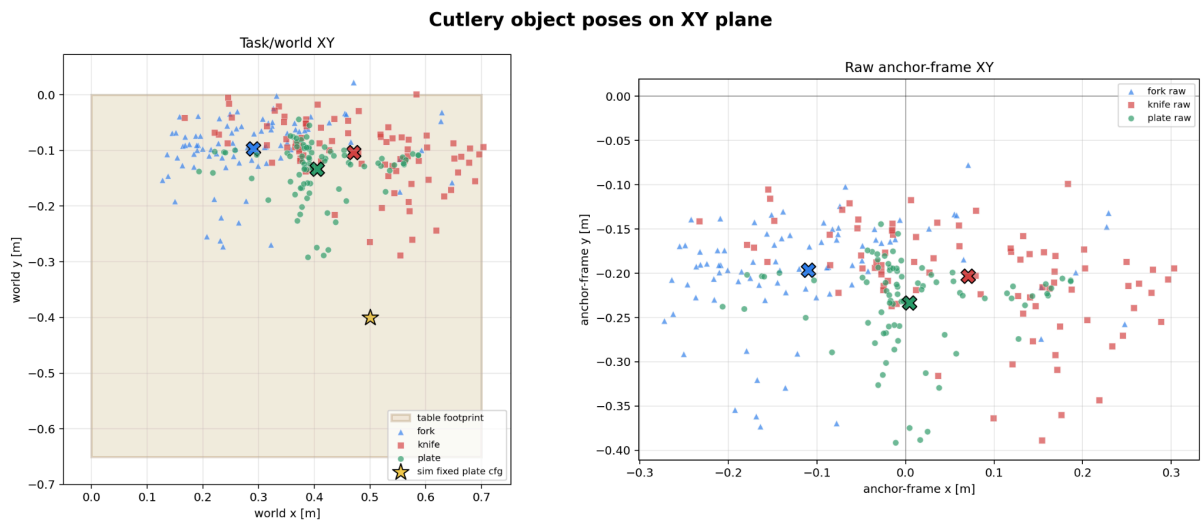


Figure 2. Extraction of objects' poses from real-world UMI collection

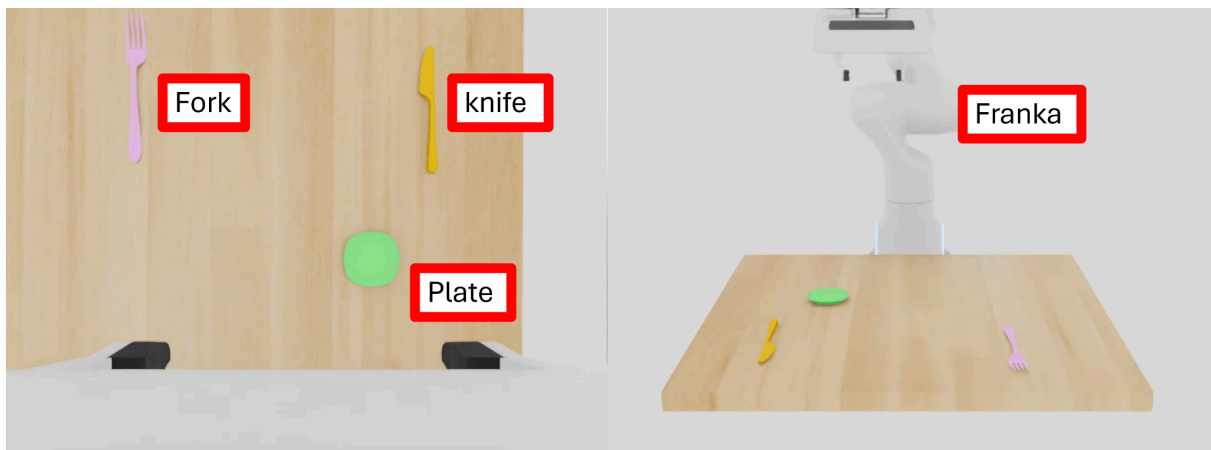


Figure 3. Isaac Sim cutlery arrangement scene

B. Simulation Data Collection in Isaac-Sim

After UMI processing, we generated simulation demonstrations in Isaac Sim / Isaac Lab. The simulation data generation stage is responsible for producing policy-training episodes in the LeRobot dataset format. Each episode contains robot observations, robot states, action sequences, object poses, and task outcome information.

Entry-Level Simulation Data Collection

For the Entry-level cutlery arrangement task, we used the Isaac Sim environment HCIS-CutleryArrangement-SingleArm-v0. The simulation scene contains the Franka robot, the dining table, a fixed plate, a fork, and a knife.

All Entry-level demonstrations were generated using the finite-state-machine-based planner. We did not use keyboard teleoperation for the final Entry-level training datasets since we found that using keyboard teleoperation to collect data in the Isaac Sim will encounter latency problems, which makes it hard to collect data. The FSM planner decomposes the task into structured manipulation stages, including approaching the utensil, grasping it, lifting it, moving it to the correct side of the plate, releasing it, and returning to a safe pose.

Only successful FSM rollouts were recorded as valid training episodes. Failed generations, such as failed grasping, dropped objects, invalid final placement, or collision-induced failures, were excluded from the final dataset. This filtering step is important because imitation learning directly learns from the demonstration distribution, so including failed demonstrations may lead to unstable or incorrect policy behavior.

The Entry-level data generation statistics are summarized below.

Dataset	Number of poses	Success episodes	Success Rate
Origin UMI	84	62	73.8%
Augmented	300	221	73.7%

Table 2. Summary of Entry-level data generation statistics

Although the two object-pose sets have similar FSM generation success rates, the 300-pose setting produces many more valid training demonstrations. The 84-pose setting produced 62 successful episodes, while the 300-pose setting produced 221 successful episodes. Therefore, the main benefit of the 300-pose dataset is not a higher FSM generation success rate, but a larger and more

diverse object pose set. We only applied the FSM to conduct data collection in the simulator.

Entry-Level Data Generation FSM Design

For the Entry-level cutlery arrangement task, we modified the original course-provided cutlery arrangement FSM. The original FSM follows a structured pick-and-place design. For each object, the robot executes seven phases: moving above the object, approaching the object, closing the gripper, lifting the object, moving above the placement target, lowering to release, and retreating upward. This design provides a clear and reproducible way to generate demonstrations for the fork-and-knife arrangement task.

Our modified FSM keeps the same overall task structure and action interface, but introduces several changes to improve demonstration quality and robustness.

First, we modified the motion timing and target heights. The hover, lift, and release heights were adjusted to make the generated trajectories more stable for thin cutlery objects. Forks and knives are difficult to grasp because small end-effector errors can cause unstable contact or missed grasps. Increasing the hover and lift margins helps reduce collision risk during transport, while adjusting the release height improves placement stability near the plate.

Second, we fixed the interpolation behavior between objects. In the original implementation, the initial end-effector position for smooth interpolation was only captured at the beginning of the first object motion. In our modified version, the end-effector position is re-captured at the beginning of every hover phase. This ensures that both the knife trajectory and the fork trajectory start with smooth motion interpolation, instead of only smoothing the first pick operation. This modification reduces discontinuities when the FSM switches from the first object to the second object.

Third, we improved the lift phase. Instead of directly following the moving object pose during lifting, the modified FSM locks the end-effector XY position at the beginning of the lift phase. This prevents the target position from drifting with the held object and produces a more stable vertical lifting behavior. This is especially important for cutlery because the object may be slightly offset from the gripper after grasping.

Fourth, we added DART-style target perturbation during data generation. At the beginning of each phase, the FSM samples a fixed Cartesian noise offset for the target position. This offset is then linearly decayed to zero throughout the phase. Therefore, the robot initially moves with a small perturbation and gradually corrects back to the nominal target. This produces

“drift-then-correct” recovery trajectories rather than perfectly deterministic demonstrations. The purpose is to reduce covariate shift during imitation learning: during policy rollout, the learned policy may deviate slightly from the expert trajectory, so training on recovery-like demonstrations can make the policy more robust.

The DART noise magnitude is phase-dependent. Larger XY perturbations are used in hover and move-above-placement phases to create lateral recovery behaviors, while grasp and release phases use much smaller noise because these phases are sensitive to precise end-effector alignment. The lift phase uses no XY noise because the lift target already locks the end-effector XY position. This design balances data diversity and demonstration validity.

Fifth, we strengthened the success criteria. In addition to checking whether the fork and knife are close to the plate and placed on the correct side, the modified FSM also checks the z-axis consistency between the fork, knife, and plate. This prevents trajectories from being counted as successful when the objects are near the plate in XY but are floating, stacked incorrectly, or placed at an unrealistic height. As a result, the final training dataset contains cleaner demonstrations.

Overall, the Entry-level FSM modification was designed to improve the generated training data rather than to change the task objective. The policy still learns the same cutlery arrangement task, but the demonstrations are smoother, more diverse, and more strictly filtered.

III. Policy Training

A. Model Architecture

We trained two imitation-learning policies, ACT and Diffusion Policies, and one visuomotor policy, VLA.

Action Chunking with Transformers, ACT

Action Chunking with Transformers predicts a chunk of future robot actions from the current visual observation and robot state. This architecture is suitable for long-horizon manipulation because it models temporally extended behaviors such as approaching, grasping, lifting, transferring, and releasing objects.

During inference, ACT uses temporal ensembles to combine overlapping action predictions. This helps smooth the predicted action sequence and reduces abrupt changes between consecutive actions. This property is useful

for the cutlery arrangement task because stable grasping and accurate placement require temporally consistent control.

We used ACT as the training policy because the cutlery arrangement task is a long-horizon, visually grounded pick-and-place task. The policy must maintain temporal consistency across approach, grasp, lift, transfer, and release phases. ACT is suitable for this setting because its chunk-based action prediction can model multi-step manipulation primitives more naturally than purely single-step behavior cloning.

Diffusion Policy

Diffusion Policy formulates visuomotor action prediction as a conditional denoising process. Given visual observations and robot states, the policy generates action trajectories through iterative denoising. Compared with deterministic behavior cloning, Diffusion Policy can better represent multimodal action distributions and generate smooth action sequences.

In this project, Diffusion Policy was trained as a comparison against ACT. For the Entry-level task, both policies were evaluated on the 300-pose dataset setting.

SmolVLA

Recently, the VLA has become more and more popular in the field of robot manipulation. We also try to train SmolVLA as an additional VLA-based policy. Unlike ACT and Diffusion Policy, which mainly learn visuomotor control from image and state observations, SmolVLA follows a vision-language-action policy design. We included this experiment to explore whether a VLA-style policy can also learn the manipulation behavior from our generated demonstration dataset. SmolVLA was trained on the 84-pose dataset and evaluated through local rollout.

B. Entry-Level Training Procedure

For Entry-level experiments, we trained ACT, Diffusion Policy, and SmolVLA using the LeRobot training pipeline. The overall procedure was:

1. Generate the simulation dataset in LeRobot format.
2. Load the dataset from the local path or Hugging Face LeRobot cache.
3. Launch `lerobot-train` with the selected ACT configuration.
4. Enable image transforms for visual robustness.
5. Train the policy for the configured number of steps.
6. Save checkpoints periodically.
7. Upload or copy the final `pretrained_model` checkpoint.
8. Run local rollout evaluation in Isaac Sim.

The Entry-level training experiments are summarized below.

	Policy	Dataset used	Training episodes	Training steps
1	ACT	84 object poses	62	100,000
2	ACT	300 object poses	221	60,000 / 80,000 / 100,000
3	Diffusion	300 object poses	221	100,000
4	SmolVLA	300 object poses	221	60,000

Table 3. Summary of Entry-level training experiment

For ACT, chunk size was selected to be 100 since each FSM demonstration stage is roughly around 100 steps, allowing the model to learn a complete manipulation phase such as approach, grasp, lift, or placement. We used `n_action_steps = 1` so the policy replans after every executed action, which improves closed-loop reactivity for thin-object grasping. And `temporal_ensemble_coeff = 0.01` was used to smooth overlapping action predictions and reduce abrupt motion. The optimizer used learning rate $1.4\text{e-}5$, weight decay $1\text{e-}4$, and 1000 warmup steps to stabilize training on a limited demonstration dataset.

Key ACT hyperparameters	Numerical Value
<code>policy.chunk_size</code>	100
<code>policy.n_action_steps</code>	1
<code>policy.temporal_ensemble_coeff</code>	0.01

Table 4. Key ACT hyperparameters

For Diffusion policy, `horizon = 16` was used to predict a short future action trajectory through the denoising model. We used `n_action_steps = 8` so that multiple predicted actions are executed before replanning, producing smoother motion while still allowing periodic correction. `n_obs_steps = 2` gives the model short-term temporal context, which helps identify whether the robot is approaching, grasping, or transporting an object. The diffusion model used 100 DDPM training timesteps with epsilon prediction. The optimizer settings were kept the same as ACT.

Key Diffusion Policy hyperparameters	Numerical Value
horizon	16
n_action_steps	8
n_obs_steps	2

Table 5. Key Diffusion Policy hyperparameters

IV. Experimental Results

Our experiments were designed to answer two main questions.

First, we evaluate **whether increasing the number of object poses improves the final policy performance**. For this purpose, we compare ACT trained on the 84-pose dataset and ACT trained on the 300-pose dataset. Second, we evaluate **whether the final performance depends on policy architecture and checkpoint selection**. For this purpose, we compare ACT and Diffusion Policy trained on the same 300-pose dataset. We also compare different ACT checkpoints trained on the 300-pose dataset. All Entry-level policy results are evaluated using OJ rollout evaluation over 30 episodes.

A. Data Quality Analysis

In the data quality evaluation, object randomization is used as the primary metric. For Entry-level settings, object poses are sampled uniformly within a predefined valid range. The underlying assumption is that more uniformly randomized object positions encourage greater diversity in FSM behavior, which in turn leads to policies with improved generalization capability.

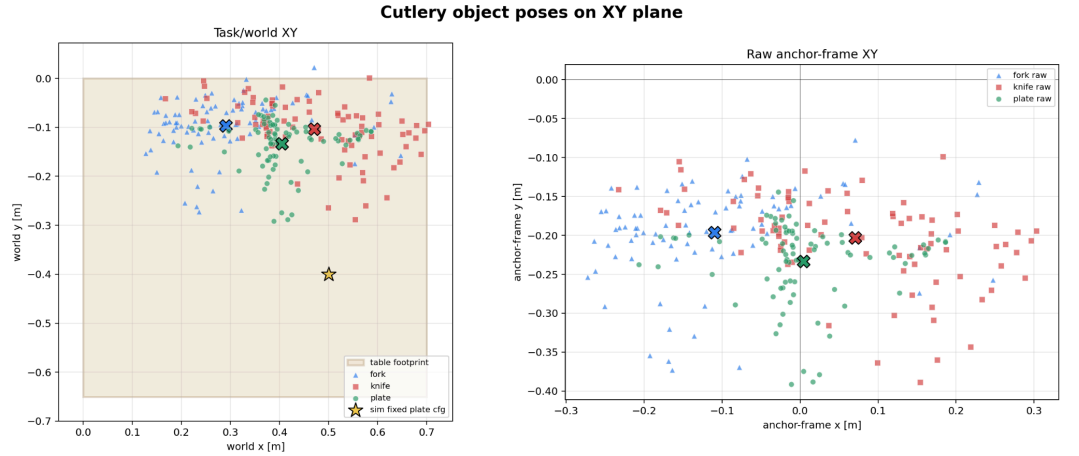


Figure 4. Entry-level object 84-pose distribution.

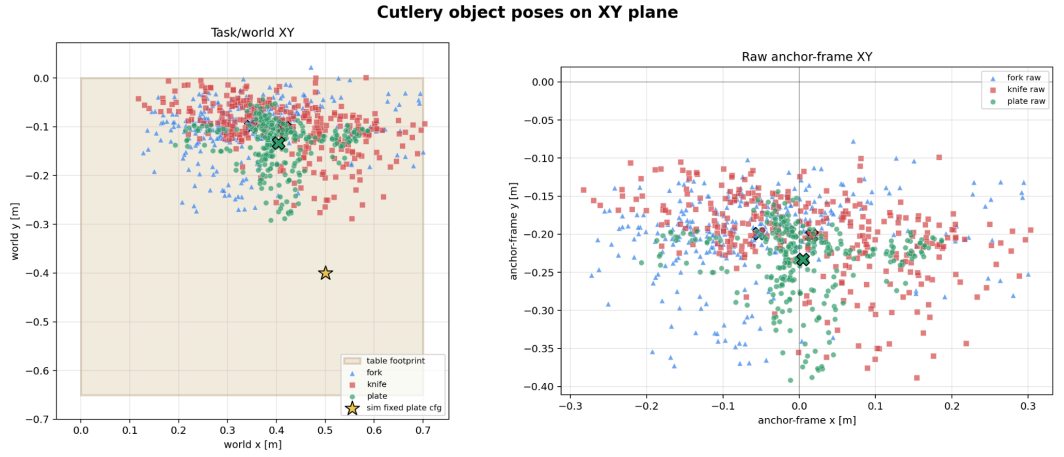
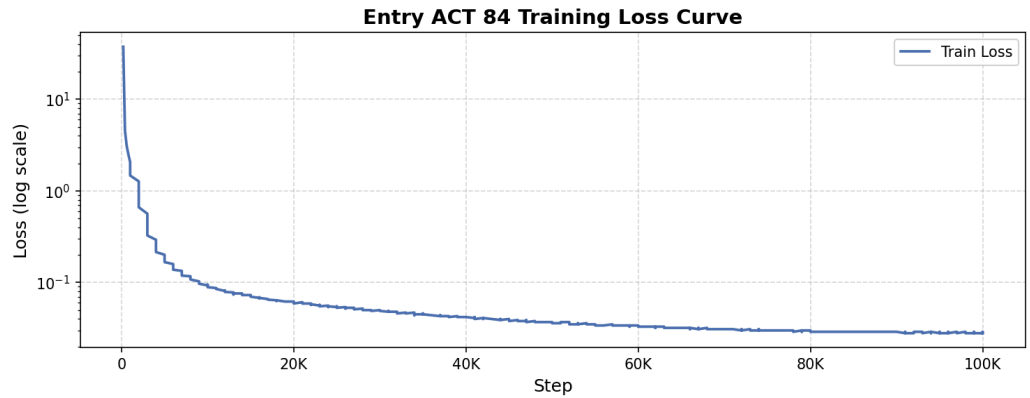
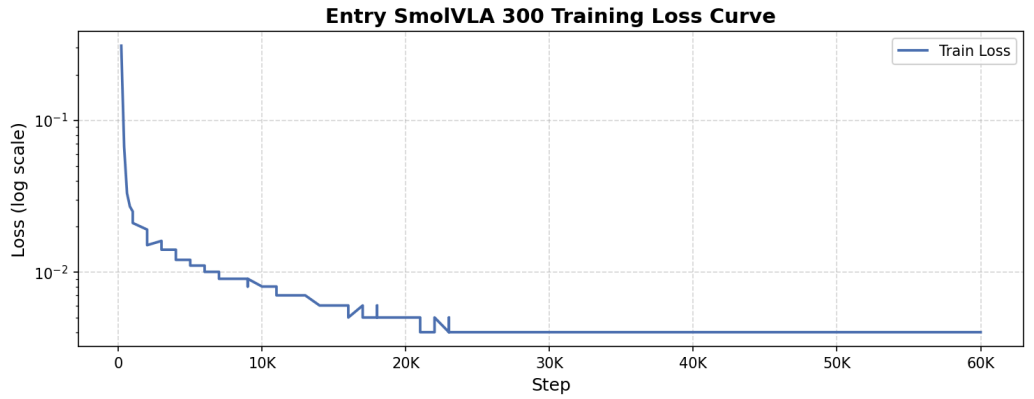
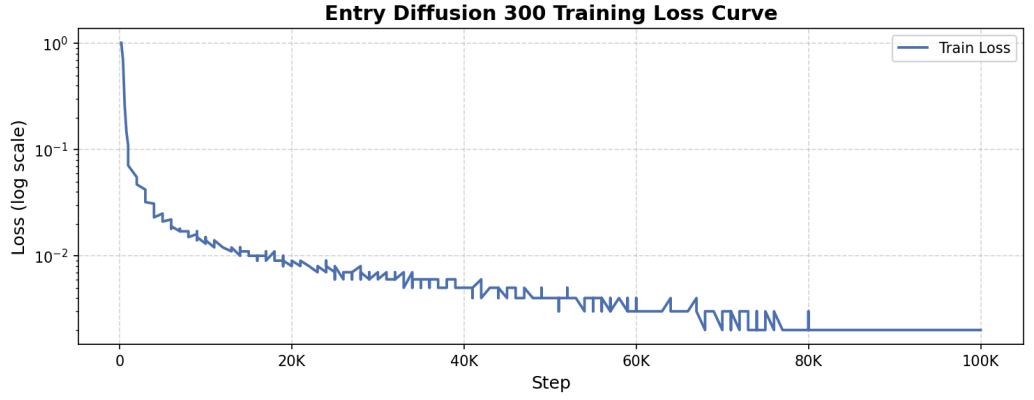
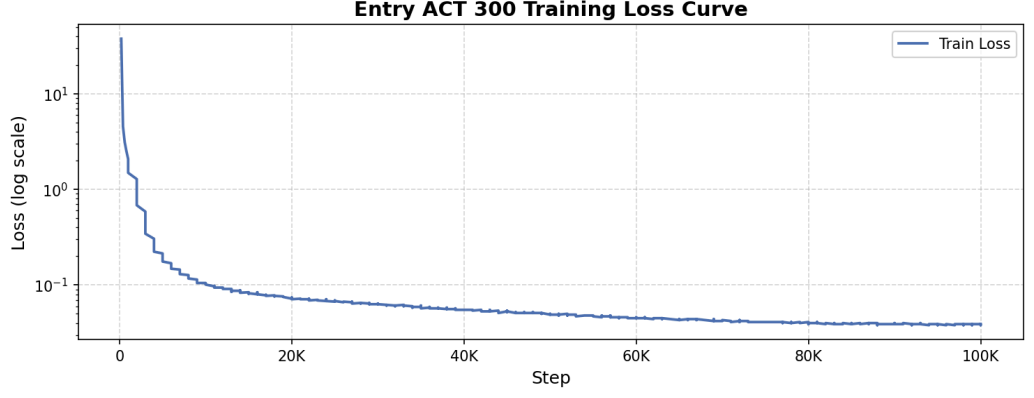


Figure 5. Entry-level object 300-pose distribution.

B. Training loss

Both ACT and Diffusion Policy perform well in terms of loss reduction. After 100k training steps, both methods converge to a loss below 0.005, exhibiting similarly fast convergence with a long-tail decay pattern. The main difference lies in the stability of the loss trajectories: ACT maintains a smoother loss curve throughout training, whereas Diffusion Policy shows noticeable fluctuations, even in the later stages of training. SmolVLA also demonstrates excellent optimization performance. Its loss converges rapidly and remains highly stable after convergence, without exhibiting the minor oscillations observed in ACT. Among the three models, SmolVLA achieves the fastest convergence and the most stable training dynamics.





C. Eval results

1) Compare different dataset size

We compared the effect of dataset size using the same ACT architecture, identical training configurations, and the same FSM-based data generation pipeline. Surprisingly, both models achieved the same success rate despite the difference in dataset size.

We hypothesize that ACT already reaches its optimal performance range with approximately 50–100 demonstrations, making additional data less beneficial. Furthermore, because both datasets were generated

using the same FSM, the increase in dataset size did not substantially improve derived trajectory diversity. As a result, the training distributions remained highly similar, and under the same number of training steps, the learned policies exhibited nearly identical performance. This may explain why scaling up the dataset did not lead to further improvements in success rate.

Policy	Dataset Size	Steps	Success rate
ACT	84	100000	43.3%
ACT	300	100000	43.3%

Table 6. Experiments for different dataset sizes

2) Compare different policy type

In this experiment, we compared different policy architectures using the same dataset and, as much as possible, the same training budget in terms of optimization steps, except for the VLA model, which required substantially longer training time.

The results show a clear performance ranking: VLA > ACT > Diffusion Policy. Beyond quantitative evaluation, we also examined the rollout behaviors of the trained policies. ACT and Diffusion Policy exhibited similar behavioral patterns. In situations where the robot failed to grasp an object—corresponding to visual states not represented in the training data—the Franka arm often became stuck oscillating in place. Consequently, many failure cases resulted from the robot remaining idle until the episode timed out.

In contrast, SmolVLA demonstrated substantially greater behavioral flexibility and robustness. The policy was able to perform actions that were never explicitly demonstrated in the training dataset. For example, when a spoon and fork overlapped, SmolVLA could adjust the gripper height to grasp the upper utensil correctly. It also showed the ability to recover from failed grasps by autonomously attempting a re-grasp, despite the fact that the FSM demonstrations did not include any corrective re-grasping behaviors. These observations suggest that the VLA model possesses stronger generalization and recovery capabilities in out-of-distribution scenarios compared to ACT and Diffusion Policy.

Policy	Dataset Size	Steps	Success rate
ACT	300	100000	43.3%
Diffusion	300	100000	36.7%
SmolVLA	300	60000	60.0%

Table 7. Experiments for different policies

3) Compare different number of steps

In the final experiment, we investigated the effect of training duration by evaluating different checkpoints from the same training run.

Surprisingly, longer training did not necessarily lead to better performance. The checkpoint at 60k training steps achieved approximately 24% higher success rate than the final 100k-step checkpoint. Based on the number of training steps and episode length, we initially estimated that 100k steps corresponded to only about 5.5 epochs, which is relatively small compared to the training regimes commonly used for many deep learning models. Therefore, we expected performance to continue improving with additional training. However, the results indicate that the optimal checkpoint was reached much earlier, around 60k steps. This finding suggests that, for our task and dataset, the performance sweet spot occurs at a lower number of effective epochs than we originally anticipated.

One possible explanation is that the policy begins to overfit to the FSM-generated demonstrations as training progresses. Although the training loss continues to decrease, additional optimization may cause the model to specialize to the narrow state distribution present in the dataset, reducing its ability to generalize during rollout. This phenomenon is particularly common in imitation learning, where improvements in behavior cloning loss do not always translate into higher task success rates. Consequently, an intermediate checkpoint may provide a better balance between fitting the demonstrations and maintaining generalization, which could explain why the 60k-step checkpoint outperformed the final 100k-step model.

Policy	Dataset Size	Steps	Success rate
ACT	300	60000	67.7%
ACT	300	80000	26.7%
ACT	300	100000	43.3%

Table 8. Experiments for different steps

V. Discussion

A. Overall Findings

The results show that imitation-learning policies can learn the basic structure of the cutlery arrangement task from FSM-generated demonstrations. However, the experiments also show that low training loss does not always lead to high rollout success. The main findings are that increasing dataset size did not improve ACT performance at 100k steps, ACT performed better than Diffusion Policy under the same setting, and the 60k-step ACT checkpoint achieved better rollout success than the 100k-step checkpoint.

B. Object Pose diversity

During the development of the Entry-level task, we experimented with a dataset in which the initial positions of the knife and fork were randomly sampled across the entire tabletop. Our original hypothesis was that increasing the diversity of object poses would improve policy generalization and ultimately lead to better performance.

However, after training, we observed the opposite effect. The resulting policy often struggled to accurately localize the utensils and frequently failed during grasping. We speculate that the degree of randomization introduced too much variability relative to the amount of available training data, making it difficult for the policy to learn a consistent mapping from visual observations to actions. Training a policy that can generalize across such a large spatial distribution may require substantially more demonstrations or more capable model architectures.

C. Limitations

There are several limitations in the current experiments. First, the evaluation used only 30 rollout episodes, so the success rate may still have variance. Second, the demonstrations were generated by an FSM planner, which produces clean but relatively narrow behavior. Third, the evaluation only

reports overall success rate and does not separately analyze grasp failure, placement error, object dropping, or timeout.

D. Future Work

Future work could focus on improving trajectory diversity and evaluation quality. The dataset should include recovery demonstrations, such as re-grasping, correcting failed grasps, and adjusting object alignment. Data aggregation could also be used to collect corrective actions from states visited by the learned policy.

In addition, future experiments should apply stronger randomization to object poses, rotations, lighting, camera viewpoints, and physical parameters. Finally, ACT should use rollout-based early stopping, Diffusion Policy should be tuned further, and SmolVLA should be evaluated under the same protocol.

For the model part, the VLA model is trained purely through trajectory imitation and does not utilize any language prompts. As a result, the model relies solely on visual observations and action demonstrations to learn the task. We believe that incorporating language instructions could further improve performance, as language provides additional semantic information about task objectives and subtask structure. By combining trajectory learning with language guidance, the VLA model may be better able to understand multi-stage tasks, generalize to unseen situations, and recover from execution failures.

VI. Work Distribution

Member	Responsibilities	Contributions
陳詩諺 112550004	Advance idea, Tune FSM, Advance scene design, Report, research&training VLA, Project Manager	18
謝嘉宸 112550019	Advanced Idea design, Data Generation, Policy Training (All policies, All levels), Rollout (All policies, All levels), Report writeup, Slides creation, Presentation recording.	18
徐瑋晨 112550047	Object poses analysis, Advanced scene config design, Advance Idea & Proposal, Advanced FSM design, Advanced-Level rollout & evaluation design, Report & Submission Documents.	18
蔡烝旭 112550099	Data Collection, Data generation (small case), Policy training(ACT, Diffusion), Joker Rollout (ACT, Diffusion), Report, Slides Design	18

游翔宇 112550103	Data Collection, Advanced Idea propose, Presentation Slides Draft, Environment Setup	10
孫傳康 112550176	Human demonstration data processing, FSM data generation, FSM Tuning, Entry-Level ACT policy training and Rollout.	18